

WEEK 1 MODULE

1. STAR SQUARE

```
int main() {  
    int n = 5;  
    for (int i = 1; i <= n; i++) { [for outer  
                                    loop]  
        for (int j = 1; j <= n; j++)  
            cout << " * ";  
        cout << endl;  
    }  
    return 0;  
}
```

```
atkins_square.cpp, 11 (31) { Star_Square }  
Enter the size of the square: 5  
*****  
*****  
*****  
*****  
*****
```

2. Square Hollow

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
```

```
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n)
                cout << "* ";
            else
                cout << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Logic Used :

- There are two nested loops
 - 1st one is for outer loop → control rows(i)
 - 2nd is for inner loop → control columns(j)
- Print * only on the boundary positions:
 - First row → $i == 1$
 - Last row → $i == n$
 - First column → $j == 1$
 - Last column → $j == n$
- Print a space (" ") in all other position

```
Star_Patterns_HollowSquare : if ($?) { .\Star_Patterns_Hollow
Enter the size of the square: 5
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

3. Triangle

```
*  
* *  
* * *  
* * * *  
* * * * *
```

CODE

```
#include <iostream>  
using namespace std;  
int main() {  
    int n=5;  
    for (int i=1; i<n; i++) {  
        for (int j=1; j<=i; j++) {  
            cout << " *";  
        }  
        cout << endl;  
    }  
    return 0;  
}
```

Logic

- Use two nested loops
 - Outer loop (i) → controls the row
 - Inner loop (j) → prints stars in each row
- In each row,
Print star equal to the row-number.

```
_Patterns_Triangle } ; if ($?) { .\Star_Patterns_T  
Enter the size of the triangle: 5  
*  
* *  
* * *  
* * * *  
* * * * *
```

4. Inverted Triangle

```
* * * * *
 * * * *
  * * *
   * *
    *
```

(CODE)

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << " * ";
        }
        cout << endl;
    }

    return 0;
}
```

LOGIC

There are two nested loops;

→ Outer loop (i) → controls the rows and runs from n down to 1.

Inner loop (j) → print stars in each row

```
Star_Patterns_InvertedTriangle ; if ($?) {
Enter the size of the inverted triangle: 5
* * * * *
 * * * *
  * * *
   * *
    *
```


5. Pyramid

```

      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *

```

CODE

```

#include <iostream>
using namespace std;
int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++)
            cout << " ";
        for (int j = 1; j <= 2 * i - 1; j++)
            cout << "* ";
        cout << endl;
    }
    return 0;
}

```

Logic

- * Use three loops:
 - Outer loop (i) → controls the row
 - There will be two inner loops
 - First inner loop → prints space (n-i)
 - Second inner loop → prints stars (2*i - 1)
- * The number of space decrease by one in each row, while the number of stars increased by two.

```

C:\Users\Amit\OneDrive\Desktop\Programming\Patterns_Pyramid> if ($?) { .\Star_
Enter the size of the pyramid: 5
      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *

```

Diamond



```
CODE
#include <iostream>
using namespace std;
int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++)
            cout << " ";

```

```
        for (int j = 1; j <= 2 * i - 1; j++)
            cout << "* ";
        cout << endl;
    }
    return 0;
}
```

Logic

A diamond pattern is made by combining

- Upper Pyramid
- Lower Inverted Pyramid

loop are used:

- Outer loop for rows.
- First inner loop for spaces.
- Second inner loop for stars ($2 * i - 1$)

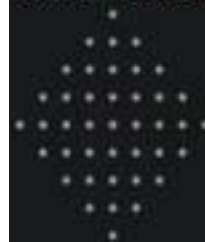
In the upper half:

- space decrease by 1
- stars increase by 2

In the lower half:

- space increase by 1
- stars decrease by 2

```
dsa\Week 1 Module - Programming Practice (3r
ns_Diamond ); if ($?) { .\Star_Patterns_Dia
Enter the size of the diamond: 5
```



7. Half Diamond

Code

```
#include <iostream>
using namespace std;

int main() {
    int n=5;
    for (int i=1; i<=n; i++) {
        for (int j=1; j<=i; j++)
            cout << " * ";
        cout << endl;
    }
    return 0;
}
```

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
*
```

Logic

A half diamond pattern is formed by combining:

→ Triangle

→ Inverted Triangle

Use two outer loops,

→ First loop prints stars from 1 to n (increasing)

→ Second loop prints stars from n-1 to 1 (decreasing)

The inner loop prints stars equal to the current row number.

```
HalfDiamond.cpp -o Star_Patterns_HalfDiamond } ; if ($?) { .
Enter the size of the half diamond: 5
```

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
*
```

3. Half Diamond Inverted

```
* * * * *
 * * * *
  * * *
   * *
    *
```

CODE

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = n; i >= 1; i--) {
```

```
        for (int j = 1; j <= i; j++)
            cout << " * ";
        cout << endl;
    }
```

Logic

- Use one outer loop that runs from n down to 1.
- Use one inner loop to print stars equal to the current row number.
- The number of stars decreased by one in each row.

```
pto\dsa\Week 1 Module - Programming Practice
cpp -o Star_Patterns_HalfDiamondInverted } ;
Enter the size of the inverted diamond: 5
```

```
* * * * *
 * * * *
  * * *
   * *
    *
```


2. Number Patterns

1 - Square

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

CODE

```
#include <iostream>
using namespace std;
int main() {
```

```
    int n=5;
    for (int i=1; i<=5; i++) {
        for (int j=1; j<=5; j++) {
            cout << "1";
        }
        cout << endl;
    }
    return 0;
}
```

Logic

- Use two nested loops;
Outer loop (i) → controls the rows
Inner loop (j) → controls the columns
- Print the same number (1) in every position of the square

```
pto\dsa\Week 1 Module - Programmi
{ .\Square }
Enter the size of the square: 5
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

2. Incrementing Pattern

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n=5;
```

```
    for (int i=1; i<=n; i++) {
```

```
        for (int j=1; j<=n; j++)
```

```
            cout << j;
```

```
        }
```

```
        cout << endl;
```

```
    }
```

Logic

- Use two nested loops:
 - Outer loop (i) → controls the rows
 - Inner loop (j) → controls the columns
- In each row, print the column numbers from 1 to n.
- The sequence 1 2 3 4 5 is repeated in every row.

pto\dsa\Week 1 Module - Progr

```
} ; if ($?) { .\Incrementing
```

Enter the number of rows: 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

3. RIGHT TRIANGLE PATTERN

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

CODE

```
#include <iostream>
using namespace std;
int main() {
    int n = 5;
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            cout << j << " ";
        }
        cout << endl;
    }
}
```

LOGIC

- Use two nested loops:
 - Outer loop (i) → controls the rows.
 - Inner loop (j) → prints numbers from 1 to the current row number (i).
- The number of elements increases by one in each row.

```
pto\dsa\Week 1 Module - Programming I
le } ; if ($?) { .\RightTriangle }
Enter the number of rows: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Inverted Diamond Pattern

```
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7
```

CODE

```
#include <iostream>
using namespace std;
int main() {
    int n = 5;
    int num = 3;
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << num << " ";
        }

```

```
        num++;
        cout << endl;
    }
    return 0;
}
```

```
pto\dsa\Week 1 Module - Programming Prac
iamond } ; if ($?) { .\InvertedDiamond }
Enter the number of rows: 5
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7
```


SANDWICH PATTERN

3 3 3 3 3

4 4 4

5

4 4 4

3 3 3 3 3

CODE

```
#include <iostream>
using namespace std;
int main()
{
    int n = 5;
    int k = 3;
    for (int i = 0; i < n - 2; i++)
    {
        for (int j = 0; j < i; j++)
        {
for (int j = 0; j < i; j++)
            cout << " ";
        }
        for (int j = 0; j < n - 2 * (i); j++)
        {
            cout << k;
        }
        k++;
        cout << endl;
    }
    k--;
```

```

for (int i = 0; i < n-3; i++) {
    k--;
    for (int j = 0; j <= i; j++)
        cout << " ";
    for (int j = 0; j <= 2 * (i+1); j++)
        cout << k;
    cout << endl;
}
return 0;
}

```

Logic

The pattern is divided into three parts:

- Upper part - print the same number with decrease count.
- Middle part - print a single number.
- Lower part - print the upper part in reverse order.

The numbers first increase towards the middle and then decrease symmetrically.

```
pto\dsa\Week 1 Module - Programming Pract  
attern } ; if ($?) { .\SandwichPattern }  
33333  
  444  
   5  
  444  
33333
```

Incrementing Diamond Pattern

```
3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3
```

CODE

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int num = 3;
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            cout << num << " ";
        }
        num++;
        cout << endl;
    }
    num = 2;
    for (int i = 4; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << num << " ";
        }
    }
}
```



```

    num--;
    cout << endl;
}
return 0;
}

```

LOGIC

- The pattern is made by combining two parts:
 - upper half → The number and its count increase by 1 in each row.
 - lower half → The number and its count decrease by 1 in each row.
- Start with a number
 - in the upper half:
 - Print the current number & items
 - Increase the number after each row.
 - in the lower half:
 - Print the current number with decreasing count.
 - Decreasing the number after each row.

```
pto\dsa\week 1 Module - Programming Pract
-o Incrementing_Diamond_Pattern } ; if (
3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3
```
